INNOMATICS RESEARCH LABS

RAG Internship Project — Final Submission

TECHNICAL DOCUMENTATION

Complete System Explanation

Design & Build a RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Submitted by: Meghna Subramani

Submission Date: April 2026

Deliverable 3 of 3

1. Introduction

1.1 What is RAG?

RAG (Retrieval-Augmented Generation) is a method used to make AI answers more accurate. Instead of answering from memory alone, the system:

1. First finds relevant information from a knowledge base
2. Then uses that information to generate the answer

> *RAG in one sentence: Find the right documents first, then generate an answer from only those documents — never from imagination.*

1.2 Why is RAG Needed?

Normal AI models have some problems:

- They can make up answers (hallucination)
- They don't know recent updates
- We can't verify where answers came from

RAG solves this:

- Answers come from real documents
- Knowledge can be updated anytime
- Responses are based on actual data

1.3 Use Case Overview

This system is built for customer support **(ShopEasy)**.

Users can ask:

- Return policy
- Shipping details
- Payment methods
- Account help

The system:

- Finds answers from a support document
- Generates a response
- Sends to human if unsure

2. System Architecture Explanation

2.1 Two Main Pipelines

The system has two parts:
1. Ingestion Pipeline (One-time setup)
- Load PDF
- Split into chunks
- Convert into vectors
- Store in database

This prepares the knowledge base

2. Query Pipeline (Runs every time)
When user asks a question:
1. Check if query is sensitive
2. Search relevant data
3. Filter useful content
4. Generate answer
5. Check confidence
6. Answer or escalate

2.2 How Components Work Together

- Query → goes to system
- Database → gives related content
- Model → generates answer
- System → decides if answer is good

Important:
Same embedding model is used everywhere to keep results accurate.

3. Design Decisions

3.1 Why 500-character chunks?

- Small chunks → very accurate but incomplete
- Large chunks → full context but less precise

So we chose medium size (500 chars)

- added overlap to avoid losing information

3.2 Embedding Choice

We used MiniLM model because:

- Runs locally (no cost)
- Fast and lightweight
- Good enough for this project

3.3 Retrieval Method

We use cosine similarity:

- Compares meaning of text
- Finds most similar chunks

We chose top 4 results because:

- Too few → may miss answer
- Too many → adds noise

3.4 Prompt Design

We used strict instructions like:
“Use only given context. Do not guess.”

This prevents:

- Fake answers
- Unverified information

4. Workflow Explanation

4.1 Why LangGraph?

- Break system into steps
- Control flow clearly
- Handle conditions easily

4.2 Node Responsibilities

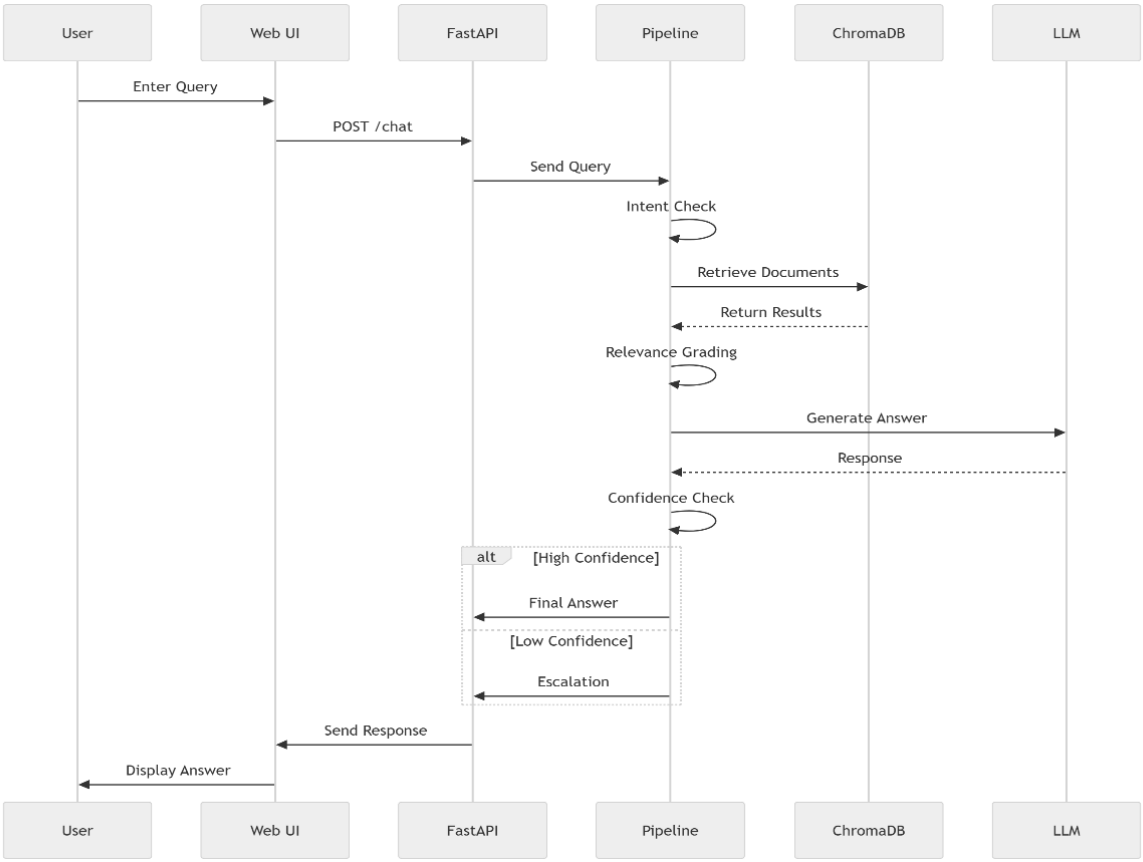
- Intent → check sensitive queries
- Retriever → fetch data
- Grader → filter data
- Generator → create answer
- HITL → handle human cases

4.3 State Management

All data is stored in a shared state object:

- Each step updates only what it needs
- Data flows step by step
- Conversation history is preserved

4.4 Sequence Diagram



5. Conditional Logic

5.1 Intent Detection

Instead of using AI, we use simple keywords:

Why?

- Faster
- Cheaper
- Enough for sensitive cases

Examples:

- “fraud”, “hacked”, “legal”

> *Design principle: Use the cheapest possible signal first. Save expensive LLM calls for tasks that actually require language understanding.*

5.2 Routing Decisions

System decides next step like this:

- Sensitive → Human
- No data → Human
- Low confidence → Human
- Otherwise → Answer

---

6. HITL Implementation

6.1 Why Human Help is Needed

System is not perfect. So:

- Complex queries → human
- Missing info → human
- Risky topics → human

6.2 Benefits

- No fake answers
- Safer responses
- Better user trust

6.3 Limitations

- Confidence check is basic
- Human response takes time
- No real-time human integration yet

7. Challenges & Trade-offs

Challenge	Decision Made	Why
Accuracy vs Speed	top_k=4	Balance
Chunk size	500 chars	Best middle
Cost vs performance	Llama3	Free + fast
DB choice	ChromaDB	Simple
Intent detection	Keywords	Fast

8. Testing Strategy

8.1 Testing Approach

- Unit tests → test each step
- Full testing → test complete system

8.2 Sample Queries and Expected Behavior

Query	Expected Path	Expected Answer Type
What is your return policy?	RAG → Answer	Direct answer: 30-day return window with steps
How long does shipping take?	RAG → Answer	Lists standard/express/overnight options
Do you accept PayPal?	RAG → Answer	Yes — PayPal and Apple Pay accepted
How do I reset my password?	RAG → Answer	Click Forgot Password on login page
Tell me about quantum pricing	RAG → HITL	Escalated — no relevant KB content found
I want to sue your company	Keyword → HITL	Escalated immediately — no retrieval attempted
I think this is fraud	Keyword → HITL	Escalated immediately — sensitive keyword
My account was hacked	Keyword → HITL	Escalated immediately — security incident

9. Future Enhancements

9.1 Multi-Document Support

Support more PDFs

9.2 Feedback Loop

Improve from human responses

9.3 Memory Integration

Support conversations

9.4 Improved Confidence Scoring

Use trained model instead of rules

9.5 Production Deployment

- Docker
- Cloud DB
- Monitoring